

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010

OPERATING SYSTEM

WEEK 10 LECTURE1

PROCESSES

WISH YOU ALL THE BEST

Wait queues

- ★ **A wait queue represents a set of sleeping processes, which are woken up by the kernel when some condition becomes true.**
- ★ **A process must often wait for some event to occur, such as for a disk operation to terminate, a system resource to be released, or a fixed interval of time to elapse.**
- ★ **Wait queues implement conditional waits on events: a process wishing to wait for a specific event places itself in the proper wait queue and relinquishes control.**
- ★ **Wait queues have several uses in the kernel, particularly for interrupt handling, process synchronization, and timing.**

Wait queues Implementation

- * Wait queues are implemented as doubly linked lists whose elements include pointers to process descriptors.
- * Each wait queue is identified by a wait queue head, a data structure of type `wait_queue_head_t`:

```
struct __wait_queue_head {  
    spinlock_t lock;  
    struct list_head task_list; };  
typedef struct __wait_queue_head wait_queue_head_t;
```

- * Because wait queues are modified by interrupt handlers as well as by major kernel functions, the doubly linked lists must be protected from concurrent accesses, which could induce unpredictable results.
- * Synchronization is achieved by the lock spin lock in the wait queue head.
- * The `task_list` field is the head of the list of waiting processes.

Wait queues Implementation

- ★ Elements of a wait queue list are of type **wait_queue_t**:

```
struct __wait_queue {  
    unsigned int flags;  
    struct task_struct * task;  
    wait_queue_func_t func;  
    struct list_head task_list; };  
typedef struct __wait_queue wait_queue_t;
```

- ★ Each element in the wait queue list represents a sleeping process, which is waiting for some event to occur; its descriptor address is stored in the task field.
- ★ The task_list field contains the pointers that link this element to the list of processes waiting for the same event.
- ★ flags field indicate kinds of sleeping process
- ★ func field specifies how the processes sleeping in the wait queue should be woken up.

WAKING UP SLEEPING PROCESSES

- ❖ **THUNDERING HERD PROBLEM**
- ❖ **Consider the following situation :**
- ❖ **Processes are all sleeping in the wait queue waiting for the SAME EVENT to occur or SAME RESOURCE to become available.**
- ❖ **When that EVENT occurs or RESOURCE becomes available, all the sleeping processes in the wait queue waiting for that event to occur are woken up**
- ❖ **Now all these processes will try to claim their right to access the RESOURCE**

WAKING UP SLEEPING PROCESSES

- ★ It is not always convenient to wake up all sleeping processes in a wait queue.
- ★ For instance, if two or more processes are waiting for exclusive access to some resource to be released, it makes sense to wake up just one process in the wait queue.
- ★ This process takes the resource, while the other processes continue to sleep.
- ★ This avoids a problem known as the "thundering herd," with which multiple processes are awakened only to race for a resource that can be accessed by one of them, with the result that remaining processes must once more be put back to sleep.

WAKING UP SLEEPING PROCESSES

❑ KINDS OF SLEEPING PROCESSES

- ★ There are two kinds of sleeping processes:
 - **Exclusive processes** (denoted by the value 1 in the flags field of the corresponding wait queue element) are selectively woken up by the kernel
 - **Non-exclusive processes** (denoted by the value 0 in the flags field) are always woken up by the kernel when the event occurs.

WAKING UP SLEEPING PROCESSES

- A process waiting for a resource that can be granted to just one process at a time is a typical exclusive process.
- Processes waiting for an event that may concern any of them are nonexclusive.
- Consider, for instance, a group of processes that are waiting for the termination of a group of disk block transfers: as soon as the transfers complete, all waiting processes must be woken up.
- The **func field** of a wait queue element is used to specify how the processes sleeping in the wait queue should be woken up.

PROCESSES

- * A process wishing to wait for a specific condition can invoke any of the functions shown in the following list.
- * The **sleep_on()** function operates on the current process: The function sets the state of the current process to **TASK_UNINTERRUPTIBLE** and inserts it into the specified wait queue. Then it invokes the scheduler, which resumes the execution of another process. When the sleeping process is awakened, the scheduler resumes execution of the **sleep_on()** function, which removes the process from the wait queue.

The **interruptible_sleep_on()** function is identical to **sleep_on()**, except that it sets the state of the current process to **TASK_INTERRUPTIBLE** instead of setting it to **TASK_UNINTERRUPTIBLE**, so that the process also can be woken up by receiving a signal.

The **sleep_on_timeout()** and **interruptible_sleep_on_timeout()** functions are similar to the previous ones, but they also allow the caller to define a time interval after which the process will be woken up by the kernel.

To do this, they invoke the **schedule_timeout()** function instead of **schedule()**

The **prepare_to_wait()**, **prepare_to_wait_exclusive()**, and **finish_wait()** functions, offer yet another way to put the current process to sleep in a wait queue. The **prepare_to_wait()** and **prepare_to_wait_exclusive()** functions set the process state to the value passed as the third parameter, then set the exclusive flag in the wait queue element respectively to 0 (nonexclusive) or 1 (exclusive)

□ Process Resource Limits

- ★ Each process has an associated set of resource limits , which specify the amount of system resources it can use.
- ★ These limits keep a user from overwhelming the system (its CPU, disk space, and so on).
- ★ The resource limits for the current process are stored in the **current->signal->rlim** field, that is, in a field of the **process's signal descriptor**
- ★ The field is an array of elements of type struct rlimit, one for each resource limit:

```
struct rlimit {      unsigned long rlim_cur;  
                    unsigned long rlim_max;  };
```

Table - Resource limits

<u>Field name</u>	<u>Description</u>
-------------------	--------------------

RLIMIT_AS

- * The maximum size of process address space, in bytes. The kernel checks this value when the process uses `malloc( )` or a related function to enlarge its address space

RLIMIT_CORE

- * The maximum core dump file size, in bytes. The kernel checks this value when a process is aborted, before creating a core file in the current directory of the process. If the limit is 0, the kernel won't create the file.

RLIMIT_CPU

- * The maximum CPU time for the process, in seconds. If the process exceeds the limit, the kernel sends it a `SIGXCPU` signal, and then, if the process doesn't terminate, a `SIGKILL` signal.

Table - Resource limits

<u>Field name</u>	<u>Description</u>
-------------------	--------------------

RLIMIT_DATA

- * The maximum heap size, in bytes. The kernel checks this value before expanding the heap of the process

RLIMIT_FSIZE

- * The maximum file size allowed, in bytes. If the process tries to enlarge a file to a size greater than this value, the kernel sends it a SIGXFSZ signal.

RLIMIT_LOCKS

- * Maximum number of file locks (currently, not enforced).

RLIMIT_MEMLOCK

- * The maximum size of nonswappable memory, in bytes. The kernel checks this value when the process tries to lock a page frame in memory using the mlock() or mlockall() system calls

Table - Resource limits

<u>Field name</u>	<u>Description</u>
-------------------	--------------------

RLIMIT_MSGQUEUE

- * Maximum number of bytes in POSIX message queues

RLIMIT_NOFILE

- * The maximum number of open file descriptors . The kernel checks this value when opening a new file or duplicating a file descriptor

RLIMIT_NPROC

- * The maximum number of processes that the user can own

RLIMIT_RSS

- * The maximum number of page frames owned by the process (currently, not enforced).

RLIMIT_SIGPENDING

- * The maximum number of pending signals for the process

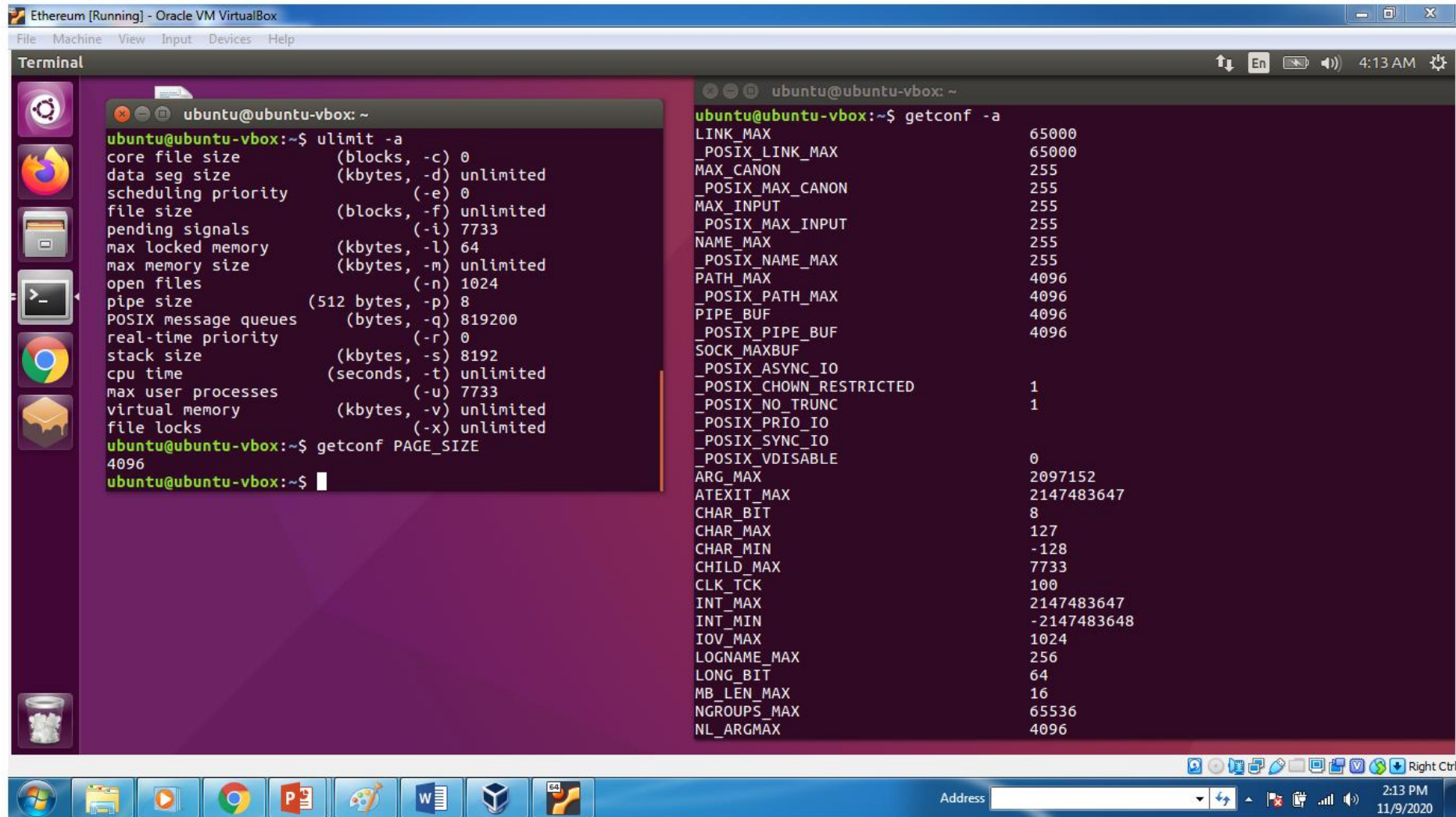
RLIMIT_STACK

- * The maximum stack size, in bytes. The kernel checks this value before expanding the User Mode stack of the process

Table - Resource limits

<u>Field name</u>	<u>Description</u>
RLIMIT_AS	The maximum size of process address space, in bytes.
RLIMIT_CORE	The maximum core dump file size, in bytes.
RLIMIT_CPU	The maximum CPU time for the process, in seconds.
RLIMIT_DATA	The maximum heap size, in bytes.
RLIMIT_FSIZE	The maximum file size allowed, in bytes
RLIMIT_LOCKS	Maximum number of file locks
RLIMIT_MEMLOCK	The maximum size of nonswappable memory, in bytes.
RLIMIT_MSGQUEUE	Maximum number of bytes in POSIX message queues
RLIMIT_NOFILE	The maximum number of open file descriptors
RLIMIT_NPROC	The maximum number of processes that the user can own
RLIMIT_RSS	The maximum number of page frames owned by the process
RLIMIT_SIGPENDING	The maximum number of pending signals for the process
RLIMIT_STACK	The maximum stack size, in bytes.

Resource limits



The screenshot shows an Oracle VM VirtualBox window titled "Ethereum [Running] - Oracle VM VirtualBox". Inside the window is a terminal window titled "Terminal" with a dark purple background. The terminal shows the output of two commands: `ulimit -a` and `getconf -a`.

```
ubuntu@ubuntu-vbox:~$ ulimit -a
core file size          (blocks, -c) 0
data seg size           (kbytes, -d) unlimited
scheduling priority     (-e) 0
file size               (blocks, -f) unlimited
pending signals         (-i) 7733
max locked memory       (kbytes, -l) 64
max memory size         (kbytes, -m) unlimited
open files              (-n) 1024
pipe size               (512 bytes, -p) 8
POSIX message queues    (bytes, -q) 819200
real-time priority      (-r) 0
stack size              (kbytes, -s) 8192
cpu time                (seconds, -t) unlimited
max user processes      (-u) 7733
virtual memory          (kbytes, -v) unlimited
file locks              (-x) unlimited
ubuntu@ubuntu-vbox:~$ getconf PAGE_SIZE
4096
ubuntu@ubuntu-vbox:~$
```

```
ubuntu@ubuntu-vbox:~$ getconf -a
LINK_MAX                65000
_POSIX_LINK_MAX         65000
MAX_CANON               255
_POSIX_MAX_CANON        255
MAX_INPUT               255
_POSIX_MAX_INPUT        255
NAME_MAX                255
_POSIX_NAME_MAX         255
PATH_MAX                4096
_POSIX_PATH_MAX         4096
PIPE_BUF                4096
_POSIX_PIPE_BUF         4096
SOCK_MAXBUF             0
_POSIX_ASYNC_IO         1
_POSIX_CHOWN_RESTRICTED 1
_POSIX_NO_TRUNC          1
_POSIX_PRIO_IO           0
_POSIX_SYNC_IO          0
_POSIX_VDISABLE          0
ARG_MAX                 2097152
ATEXIT_MAX              2147483647
CHAR_BIT                8
CHAR_MAX                127
CHAR_MIN                -128
CHILD_MAX               7733
CLK_TCK                 100
INT_MAX                 2147483647
INT_MIN                 -2147483648
IOV_MAX                 1024
LOGNAME_MAX             256
LONG_BIT                64
MB_LEN_MAX              16
NGROUPS_MAX             65536
NL_ARGMAX               4096
```

❑ Process Switch

- ★ To control the execution of processes, the kernel must be able to suspend the execution of the process running on the CPU and resume the execution of some other process previously suspended.
- ★ This activity goes variously by the names **process switch, task switch, or context switch.**

❑ Hardware Context

- ★ While each process can have its own address space, all processes have to share the CPU registers.
- ★ So before resuming the execution of a process, the kernel must ensure that each such register is loaded with the value it had when the process was suspended.
- ★ The set of data that must be loaded into the registers before the process resumes its execution on the CPU is called the hardware context .
- ★ The hardware context is a subset of the process execution context, which includes all information needed for the process execution.
- ★ In Linux, a part of the hardware context of a process is stored in the process descriptor, while the remaining part is saved in the Kernel Mode stack.

PROCESS SWITCH

- ❑ **Task State Segment**
- ★ **Linux uses hardware support offered by the 80x86 architecture and performs a process switch through jmp instruction to the selector of the Task State Segment Descriptor of the next process**
- ★ **The 80x86 architecture includes a specific segment type called the Task State Segment (TSS), to store hardware contexts.**
- ★ **Linux kernel sets up a TSS for each distinct CPU in the system.**
- ★ **This is done for two main reasons:**
 - **When an 80x86 CPU switches from User Mode to Kernel Mode, it fetches the address of the Kernel Mode stack from the TSS.**
 - **When a User Mode process attempts to access an I/O port by means of an in or out instruction, the CPU may need to access an I/O Permission Bitmap stored in the TSS to verify whether the process is allowed to address the port.**

❑ Task State Segment

- * More precisely, when a process executes an in or out I/O instruction in User Mode, the control unit performs the following operations:
 - It checks the 2-bit IOPL field in the eflags register. If it is set to 3, the control unit executes the I/O instructions. Otherwise, it performs the next check.
 - It accesses the tr register to determine the current TSS, and thus the proper I/O Permission Bitmap.
 - It checks the bit of the I/O Permission Bitmap corresponding to the I/O port specified in the I/O instruction. If it is cleared, the instruction is executed; otherwise, the control unit raises a "General protection " exception.
- * The **tss_struct** structure describes the format of the TSS.
- * The **init_tss** array stores one TSS for each CPU on the system.
- * At each process switch, the kernel updates some fields of the TSS so that the corresponding CPU's control unit may safely retrieve the information it needs.
- * Thus, the TSS reflects the privilege of the current process on the CPU, but there is no need to maintain TSSs for processes when they're not running.

❑ Task State Segment

- ★ Each TSS has its own 8-byte Task State Segment Descriptor (TSSD).
- ★ This descriptor includes a 32-bit Base field that points to the TSS starting address and a 20-bit Limit field.
- ★ The S flag of a TSSD is cleared to denote the fact that the corresponding TSS is a System Segment
- ★ The Type field is set to either 9 or 11 to denote that the segment is actually a TSS.
- ★ The TSSDs created by Linux are stored in the Global Descriptor Table (GDT), whose base address is stored in the gdtr register of each CPU.
- ★ The **tr register** of each CPU contains the TSSD Selector of the corresponding TSS.
- ★ The register also includes two hidden, nonprogrammable fields: the Base and Limit fields of the TSSD. In this way, the processor can address the TSS directly without having to retrieve the TSS address from the GDT.

PROCESS SWITCH

❑ thread field

- ★ At every process switch, the hardware context of the process being replaced must be saved somewhere. It cannot be saved on the TSS, as in the original Intel design, because Linux uses a single TSS for each processor, instead of one for every process.
- ★ Thus, each process descriptor includes a field called **thread of type `thread_struct`**, in which the kernel saves the hardware context whenever the process is being switched out.
- ★ This data structure includes fields for most of the CPU registers, except the general-purpose registers such as `eax`, `ebx`, etc., which are stored in the Kernel Mode stack.

□ Performing the Process Switch

- ★ A process switch may occur at just one well-defined point: the **schedule()** function, using how the kernel performs a process switch.
- ★ Essentially, every process switch consists of two steps:
 - ★ Switching the Page Global Directory to install a new address space;
 - ★ Switching the Kernel Mode stack and the hardware context, which provides all the information needed by the kernel to execute the new process, including the CPU registers.
- ★ Again, we assume that prev points to the descriptor of the process being replaced, and next to the descriptor of the process being activated. prev and next are local variables of the schedule() function.
- ★ The second step of the process switch is performed by the **switch_to macro**. It is one of the most hardware-dependent routines of the kernel

PROCESS SWITCH

- * The 80x86 microprocessors do not automatically save the FPU, MMX, and XMM registers in the TSS.
- * However, they include some hardware support that enables kernels to save these registers only when needed.
- * The hardware support consists of a TS (Task-Switching) flag in the cr0 register, which obeys the following rules:
 - Every time a hardware context switch is performed, the TS flag is set.
 - Every time an ESCAPE, MMX, SSE, or SSE2 instruction is executed when the TS flag is set, the control unit raises a "Device not available " exception
- * The TS flag allows the kernel to save and restore the FPU, MMX, and XMM registers only when really needed.

CREATING PROCESSES

❑ Creating Processes

- ✱ Unix operating systems rely heavily on process creation to satisfy user requests.
- ✱ For example, the shell creates a new process that executes another copy of the shell whenever the user enters a command.
- ✱ Traditional Unix systems treat all processes in the same way: resources owned by the parent process are duplicated in the child process.
- ✱ This approach makes process creation very slow and inefficient, because it requires copying the entire address space of the parent process.
- ✱ The child process rarely needs to read or modify all the resources inherited from the parent; in many cases, it issues an immediate **execve()** and wipes out the address space that was so carefully copied.

CREATING PROCESSES

- * Modern Unix kernels solve the problem of duplicating address space by introducing three different mechanisms:
- * **Copy On Write technique (fork() system call)**
 - allows both the parent and the child to read the same physical pages. Whenever either one tries to write on a physical page, the kernel copies its contents into a new physical page that is assigned to the writing process.
- * **Lightweight processes (Clone() system call)**
 - allow both the parent and the child to share many per-process kernel data structures, such as the paging tables (and therefore the entire User Mode address space), the open file tables, and the signal dispositions.
- * **vfork() system call**

creates a process that shares the memory address space of its parent. To prevent the parent from overwriting data needed by the child, the parent's execution is blocked until the child exits or executes a new program.

clone() System Call

Lightweight processes are created in Linux by using a function named clone(), which uses the following parameters:

- fn** Specifies a function to be executed by the new process; when the function returns, the child terminates. The function returns an integer, which represents the exit code for the child process.
- arg** Points to data passed to the fn() function.
- flags** Miscellaneous information. The low byte specifies the signal number to be sent to the parent process when the child terminates; the SIGCHLD signal is generally selected.
- child_stack** Specifies the User Mode stack pointer to be assigned to the esp register of the child process. The invoking process (the parent) should always allocate a new stack for the child.
- tls** Specifies the address of a data structure that defines a Thread Local Storage segment for the new lightweight process. Meaningful only if the CLONE_SETTLS flag is set.
- ptid** Specifies the address of a User Mode variable of the parent process that will hold the PID of the new lightweight process. Meaningful only if the CLONE_PARENT_SETTID flag is set.
- ctid** Specifies the address of a User Mode variable of the new lightweight process that will hold the PID of such process. Meaningful only if the CLONE_CHILD_SETTID flag is set.

do_fork() function

- ★ The **do_fork()** function, which handles the **clone()**, **fork()**, and **vfork()** system calls
- ★ **do_fork()** makes use of an auxiliary function called **copy_process()** to set up the process descriptor and any other kernel data structure required for child's execution.
- ★ Invokes **copy_process()** to make a copy of the process descriptor. If all needed resources are available, this function returns the address of the **task_struct** descriptor just created.

copy_process() function

- ★ The **copy_process()** function sets up the process descriptor and any other kernel data structure required for a child's execution. Its parameters are the same as **do_fork()**, plus the PID of the child.

❑ Kernel Threads

- ★ Kernel threads run only in Kernel Mode, while regular processes run alternatively in Kernel Mode and in User Mode.
- ★ Because kernel threads run only in Kernel Mode, they use only linear addresses greater than PAGE_OFFSET.
- ★ Regular processes, on the other hand, use all four gigabytes of linear addresses, in either User Mode or Kernel Mode.

❑ Creating a kernel thread

- ★ The **kernel_thread()** function creates a new kernel thread.
- ★ It receives as parameters the address of the kernel function to be executed (fn), the argument to be passed to that function (arg), and a set of clone flags (flags).
- ★ This function essentially invokes **do_fork()**

□ Process 0

- * The ancestor of all processes, called process 0, the **idle process**, or, for historical reasons, the **swapper process**, is a kernel thread created from scratch during the initialization phase of Linux .
- * The `start_kernel( )` function initializes all the data structures needed by the kernel, enables interrupts, and creates another kernel thread, named process 1 (more commonly referred to as the **init process**)

`kernel_thread(init, NULL, CLONE_FS|CLONE_SIGHAND);`

- * The newly created kernel thread has PID 1 and shares all per-process kernel data structures with process 0.
- * When selected by the scheduler, the init process starts executing the `init( )` function.

Process 0

- ★ After having created the init process, process 0 executes the **cpu_idle() function**, which essentially consists of repeatedly executing the hlt assembly language instruction with the interrupts enabled.
- ★ Process 0 is selected by the scheduler only when there are no other processes in the TASK_RUNNING state.
- ★ In multiprocessor systems there is a process 0 for each CPU.
- ★ Right after the power-on, the BIOS of the computer starts a single CPU while disabling the others.
- ★ The swapper process running on CPU 0 initializes the kernel data structures, then enables the other CPUs and creates the additional swapper processes by means of the **copy_process()** function passing to it the value 0 as the new PID.
- ★ Moreover, the kernel sets the cpu field of the tHRead_info descriptor of each forked process to the proper CPU index.

□ Process 1

- ★ The kernel thread created by process 0 executes the `init( )` function, which in turn completes the initialization of the kernel.
- ★ Then `init( )` invokes the `execve( )` system call to load the executable program `init`.
- ★ As a result, the **init** kernel thread becomes a regular process having its own per-process kernel data structure.
- ★ The `init` process stays alive until the system is shut down, because it creates and monitors the activity of all processes that implement the outer layers of the operating system.

❑ Destroying Processes

- ★ Most processes "die" in the sense that they terminate the execution of the code they were supposed to run.
- ★ When this occurs, the kernel must be notified so that it can release the resources owned by the process; this includes memory, open files, semaphores, etc.
- ★ The usual way for a process to terminate is to invoke the **exit()** library function, which releases the resources allocated by the C library, executes each function registered by the programmer, and ends up invoking a system call that evicts the process from the system.
- ★ The **exit()** library function may be inserted by the programmer explicitly.
- ★ Additionally, the C compiler always inserts an **exit()** function call right after the last statement of the **main()** function.

❑ Process Termination

- ★ In Linux there are two system calls that terminate a User Mode application:
- ★ The **exit_group()** system call, which terminates a full thread group, that is, a whole multithreaded application.
- ★ The main kernel function that implements this system call is called **do_group_exit()**. This is the system call that should be invoked by the **exit()** C library function.
- ★ The **_exit()** system call, which terminates a single process, regardless of any other process in the thread group of the victim.
- ★ The main kernel function that implements this system call is called **do_exit()**. This is the system call invoked, for instance, by the **pthread_exit()** function of the LinuxThreads library.

❑ Process Removal

- * The Unix operating system allows a process to query the kernel to obtain the PID of its parent process or the execution state of any of its children.
- * A process may, for instance, create a child process to perform a specific task and then invoke some wait()-like library function to check whether the child has terminated.
- * If the child has terminated, its termination code will tell the parent process if the task has been carried out successfully.
- * **What happens if parent processes terminate before their children?**
- * In such a case, the system could be flooded with zombie processes whose process descriptors would stay forever in RAM. This problem is solved by forcing all orphan processes to become children of the init process. In this way, the init process will destroy the zombies while checking for the termination of one of its legitimate children through a wait()-like system call.
- * **The release_task() function detaches the last data structures from the descriptor of a zombie process; it is applied on a zombie process in two possible ways: by the do_exit() function if the parent is not interested in receiving signals from the child, or by the wait4() or waitpid() system calls after a signal has been sent to the parent.**

INTERRUPTS & EXCEPTIONS

□ Questions

- * What is a process?
- * What are Lightweight Processes in Linux?
- * Explain what is meant by multithreaded application. Describe how multithreaded applications are implemented in linux. Give three examples of POSIX-compliant pthread libraries.
- * Describe thread group in linux.
- * What is meant by process descriptor and explain the role of the process descriptor?
- * With the help of a schematic, describe the Linux process descriptor struct `task_struct`.
- * List the seven possible process states in Linux OS and explain.
- * Describe the following process states in linux :
 - * `TASK_RUNNING`
 - * `TASK_INTERRUPTIBLE`
 - * `TASK_UNINTERRUPTIBLE`
 - * `TASK_STOPPED`
 - * `TASK_TRACED`
 - * `EXIT_ZOMBIE`
 - * `EXIT_DEAD`
- * State the purpose of `TASK_UNINTERRUPTIBLE` state in linux process.
- * State the two approaches used by kernel to identify a process.
- * What is meant by PID? Why is it required?
- * What is the upper limit on the PID values in 32-bit architectures?
- * What is the upper limit on the PID values in 64-bit architectures?
- * State the purpose of a `pidmap_array` bitmap in linux. Indicate the size of the `pidmap_array` bitmap.
- * Indicate which lightweight process becomes by default the thread group leader. What is the PID assigned to the thread group?
- * Since kernel must be able to handle many processes at the same time, hence process descriptors are stored in memory rather than in the memory area permanently assigned to the kernel.

INTERRUPTS & EXCEPTIONS

□ Questions

- * Describe the two different data structures assigned to every process by the kernel in a single per-process memory area. Indicate the length of this memory area and explain the data structures stored in this memory area with the help of a diagram showing clearly both these data structures.
- * Explain the purpose of the following two different data structures assigned to every process by the kernel in a single per-process memory area. Indicate the size of each of these data structures.
 - a) Thread_info structure
 - b) kernel mode process stack
- * State the key benefit in terms of efficiency offered by the kernel in providing a close association between the thread_info structure and the kernel mode stack.
- * State the purpose of current macro in linux.
- * Explain what is meant by process list. Explain how process list is implemented in linux.
- * With the help of a block diagram, show the special data structures used to implement the process list. Also indicate the processes the first two nodes in the data structure point to.
- * Explain how runqueue is implemented in linux to achieve scheduler speedup to select the best runnable process in constant time.
- * With the help of a figure, illustrate the relationship between parent and siblings of a group of processes. Process P0 successively created P1, P2, and P3. Process P3, in turn, created process P4
- * Write short notes on pid hash table and chained lists.
- * With the help of block diagram, describe pid hash table and chained lists.
- * What are the four PID hash tables used in linux. Why are they required?
- * With the help of a block diagram, describe the four hash tables and show how these data structures are implemented showing clearly the chained lists and processes in each group.

INTERRUPTS & EXCEPTIONS

□ Questions

- * Explain the purpose of wait queue in linux.
- * Describe runqueue and wait queues in Linux.
- * Indicate the three important uses of the wait queues in the kernel.
- * Describe how wait queues are implemented in linux by providing the following two data structures - struct `_wait_queue_head` and struct `wait_queue` and clearly indicating the purpose of various fields in these data structures.
- * What is meant by “thundering herd” problem. Explain how it is tackled.
- * What are the two kinds of sleeping processes in wait queue. Why do you require this classification.
- * Explain process Resource limits in Linux and list the various resource limits specified.
- * State the purpose of the following commands in unix/linux
 - a) `ulimit -Ha` b) `ulimit -Sa` c) `getconf -a` d) `getconf PAGE_SIZE`
- * Describe Process switch or context switch or task switch in linux.
- * Explain Hardware context in Linux.
- * Write short notes on the following :
 - a) Task State Segment b) Thread field
- * Describe how kernel performs a process switch using `schedule()` function.
- * Explain three different mechanisms introduced by modern Unix kernels to solve the problem of duplicating address space. Also indicate the system call associated with each one of these mechanisms to create a process.
- * State the three system calls used to create a process. Why are these system calls required?
- * Write the system call used to create a lightweight process in linux.
- * Write short notes on
 - a) Process 0 b) Process 1
- * Describe the operations performed by the following functions in linux :
 - a) `do_fork()` b) `copy_process()`
- * What are kernel threads? Explain how kernel threads are created in linux.
- * State the purpose of following functions in linux.
 - `kernel_thread()` b) `copy_process()` c) `exit()` d) `exit_group()`
- * Describe how processes are created and terminated/destroyed in Linux.
- * What is orphan process? How orphan process is handled in linux to eliminate memory leak?
- * What happens if parent process terminate before its children? How this problem is solved to eliminate memory leaks?

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 3 of Text Book

Text Book

➤ Understanding the Linux Kernel

- Daniel P/ Bovet and Marco Cesati
Edition, O' Reilly Media 2005, ISBN 978-0596005658**

SOC 3010

OPERATING SYSTEM

END OF
WEEK 10 LECTURE 1

PROCESSES

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

